
AGoQ: Activation and Gradient Quantization for Memory-Efficient Distributed Training of LLMs

Wenxiang Lin¹ Juntao Huang¹ Luhan Zhang¹ Laiyi Li¹ Xiang Bao² Mengyang Zhang² Bing Wang²
Shaohuai Shi¹

Abstract

Quantization is a key method for reducing the GPU memory requirement of training large language models (LLMs). Yet, current approaches are ineffective for 4-bit activations and 8-bit gradients, which would easily cause slow convergence or accuracy loss. To address this, we introduce AGoQ, incorporating two new techniques: 1) a layer-aware activation quantization algorithm that allocates appropriate bit-widths for activations of various layers based on their types and pipeline stages to achieve near 4-bit activation storage, and 2) a gradient quantization algorithm that reduces memory usage and shortens communication time by employing 8-bit gradient storage and precision-preserving 8-bit All-Reduce communication. We conduct extensive experiments using different sizes of LLMs on two GPU clusters (up to 64 GPUs), and the experimental results show that our AGoQ reduces the memory by up to 52% and achieves up to $1.34\times$ improvement of training speed compared to state-of-the-art training systems Megatron-LM (w/ or w/o ZeRO), COAT and DeepSpeed with 8B to 32B LLaMA models, while achieving convergence loss on pretraining and comparable accuracy on downstream tasks with LLaMA architectures.

1. Introduction

Distributed training has become a de-facto approach to accelerate the training process of deep neural networks (DNNs) on multi-GPU/TPU clusters (Narayanan et al., 2021; Pan et al., 2024; Shi et al., 2024; Pan et al., 2025; Lin et al., 2025). Particularly, data parallelism (DP) has been widely used by

distributing training data to different workers (or GPUs) to train a model collaboratively (Dean et al., 2012). However, with the model size significantly increased as seen in large language models (LLMs), the memory requirement for training LLMs becomes a significant pressure (Brown et al., 2020). Thus, training LLMs typically requires using model parallelism, including tensor parallelism (TP) (Narayanan et al., 2021) and pipeline parallelism (PP) (Huang et al., 2019), which partition model parameters across different devices so that each GPU has enough memory to store the required data. DP, TP, and PP have been default features in the popular LLM training framework Megatron-LM (Narayanan et al., 2021). Another popular memory-efficient training system, DeepSpeed (Rasley et al., 2020) exploits the zero redundancy optimizer (ZeRO) series (ZeRO-1/2/3) (Rajbhandari et al., 2020; Ren et al., 2021; Rajbhandari et al., 2021) to save memory in LLM training. The newly introduced fully sharded data parallelism (FSDP) (Zhao et al., 2023) in PyTorch’s ecosystem has a similar idea to ZeRO-3.

The device memory occupation of training an LLM mainly consists of model parameters, gradients, optimizer states, and temporary activations. Among them, the activations typically occupy the largest proportion of memory and are linearly increased with the increase of sequence length and batch size, which are two common hyper-parameters. There have been extensive studies aimed at reducing the memory footprint by reducing the occupation of activations including activation recomputation or offloading (Chen et al., 2016; Yuan et al., 2024; Wu et al., 2025) and activation quantization (Evans & Aamodt, 2021; Liu et al., 2022; Xi et al., 2024; 2025; Shamshoum et al., 2025; Chen et al., 2025). Activation recomputation (or offloading) is a system-level optimization technique; thus, it has no side effects on model accuracy, but it introduces extra overhead by recomputing (or uploading) activations for backpropagation. Activation quantization, on the other hand, uses low-precision formats (e.g., INT8 (Xi et al., 2024) and FP8 (Xi et al., 2025)) to store activation values and dequantizes them back to BF16/FP16 for backpropagation. However, the quantization and dequantization processes (even when using only 8-bit INT8 or FP8) result in accuracy loss compared to pure BF16/FP16 (Xi et al., 2024; 2025). Jetfire (Xi et al., 2024)

¹School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen ²Huawei Technologies Ltd. Correspondence to: Shaohuai Shi <shaohuais@hit.edu.cn>.

and COAT (Xi et al., 2025) attempt to address the accuracy loss problem of 8-bit activation quantization by employing dynamic quantization and block-wise quantization, *but they are still not applicable to lower-bit formats (e.g., 4-bit).*

Furthermore, in terms of gradient memory, although there are extensive works (Tang et al., 2021; Bai et al., 2021; Shi et al., 2021; Peng et al., 2023a; Huang et al., 2024; Wang et al., 2024) trying to compress gradients to reduce communication overheads, the gradients are still stored in high-precision (FP32) and quantized to low-precision for communication, which means the gradients still occupy the same size of memory as model parameters. One notable study by Microsoft (Peng et al., 2023b) tries to use FP8 format for gradients by using scaling factors to preserve model convergence; however, *it still easily causes convergence slowdown in training LLMs due to the accuracy loss of gradient accumulation in FP8.*

To this end, in this work, we aim to push activation quantization and gradient quantization a step further and make them practical in LLM training. Specifically, we propose AGoQ with Activation quantization that can use approximate 4-bit precision and Gradient quantization with 8-bit for memory-efficient storage and communication-efficient collective, which are compatible with optimizer state Quantization (Dettmers et al., 2022) without sacrificing model convergence. To preserve model accuracy and improve system throughput, AGoQ is equipped with our several novel techniques: 1) a layer-aware activation quantization (LAAQ) algorithm (§4) that assigns a proper number of bits for activations of different layers according to their layer types and PP stages, which achieves near 4 bits for each element of activation, 2) a precision-preserved quantized gradient storage and communication algorithm named QuanGrad (§5) that uses 8-bit representation (FP8) to store gradients for local accumulation to save memory and for All-Reduce communication to reduce communication time. We conduct extensive experiments using different sizes (from 8B to 34B) of LLMs on a 64-GPU cluster, and the experimental results show that our AGoQ reduces memory by up to 52% and achieves a $1.34\times$ improvement in training speed without sacrificing training loss or accuracy compared to state-of-the-art training systems, including Megatron-LM (w/ or w/o ZeRO), DeepSpeed, and COAT.

2. Preliminaries

2.1. Transformer Layer

Currently, LLMs with transformer architecture (Vaswani et al., 2017) are the most popular, and a transformer is typically composed of multiple stacked transformer layers. One transformer layer consists of two main sub-components: a self-attention mechanism and a Multi-Layer Perceptron

(MLP) typically with two Feed Forward Networks (FFNs). To help easily understand our error analysis in §4.1, we illustrate their equations.

Attention The attention layer (Vaswani et al., 2017) consists of several linear layers that project the input into queries (Q), keys (K), and values (V). The scaled dot-product attention is then computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V, \quad (1)$$

where d is the dimension of the key vectors.

MLP The multi-layer perceptron (MLP) block in Transformer layers consists of two linear transformations with a non-linear activation function in between. Typically, the MLP first expands the feature dimension from M to $4M$ (or $8M$ on LLaMa models (Dubey et al., 2024)) with the first linear layer and then projects it back to the original dimension, which can be represented as:

$$\text{MLP}(X) = W_2 \times \text{actfunc}(W_1 \times X), \quad (2)$$

where $W_1 \in \mathbb{R}^{M \times 4M}$, $W_2 \in \mathbb{R}^{4M \times M}$ are weight matrices, and actfunc is an activation function like SiLU (Ramachandran et al., 2017).

SiLU The Sigmoid Linear Unit (SiLU) is a smooth, non-monotonic activation function defined as:

$$\text{SiLU}(X) = X \odot \sigma(X), \quad (3)$$

where $\odot$ denotes the Hadamard product and $\sigma(X)$ is the sigmoid function.

LayerNorm Layer normalization (LayerNorm) (Ba et al., 2016) is applied to normalize the inputs across the feature dimension for each token independently. RMSNorm (Zhang & Sennrich, 2019) is one of the most famous LayerNorm functions:

$$\text{RMSNorm}(X) = \gamma \frac{X}{\sqrt{\frac{1}{d}\|X\|_2^2 + \epsilon}}, \quad (4)$$

where $\|X\|_2^2 = \sum X_i^2$, γ is a trainable parameter, d is the number of elements of X , and ϵ is a small constant for numerical stability.

For each hidden layer, its input X must be saved during the forward pass and reused during backpropagation to compute gradients with respect to activations and, for trainable layers, weights. This requirement leads to substantial memory usage. Since different layers perform different types of computations, we observe that compressing X into a low-bit format can introduce varying and potentially large errors in the resulting gradient calculations (details in §4.1).

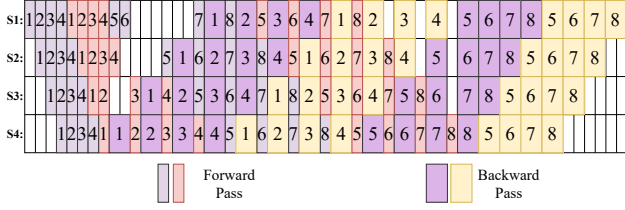


Figure 1. An example of Interleaved 1F1B PP with four stages and each mini-batch divided into eight micro-batches.

2.2. Paradigms of Parallelism

Data Parallelism (DP) distributes a mini-batch of samples to multiple workers. During backpropagation, the gradients of each worker in the same DP group are aggregated through an All-Reduce operation so that they can use the identical gradient to update the model parameters. The All-Reduce operation accumulates distributed gradients (say X_i at i worker) from all workers (say P workers) using a reduction operation (typically sum or mean in training), which can be formally represented

$$X = \text{AllReduce}(X_1, X_2, \dots, X_P) = \sum_{i=1}^P X_i. \quad (5)$$

The gradients have the same dimensionality as the model weights, which means additional memory is required to store them for communication and updating the model. Compressing the gradient using 8-bit easily causes accuracy loss due to the data overflow of the summation of AllReduce (Peng et al., 2023b).

Pipeline Parallelism (PP) (Huang et al., 2019; Narayanan et al., 2021) is a commonly used model partitioning strategy for distributed training. In PP, the layers of the model are distributed across multiple devices. For models composed of repeated transformer blocks, this typically means assigning an equal number of consecutive transformer layers to each device. To leverage parallelism within a batch, each batch is further divided into smaller mini-batches. The execution of these mini-batches is then pipelined across devices, overlapping the process of different transformer layers on different devices. To reduce the bubbles, the variant PP known as Interleaved 1F1B (Narayanan et al., 2021), as illustrated in Fig. 1, has been widely used in Megatron-LM. In this scheme, after a mini-batch passes through the entire device sequence (from the first to the last device), it is sent back to the first device and traverses the device sequence again. This requires partitioning the model into more granular segments and placing these segments evenly across devices according to the order in which the mini-batch will traverse them.

Additionally, each forward pass in Fig. 1 stores a portion of activations in GPU memory for the subsequent backward pass, while each backward pass releases part of these activa-

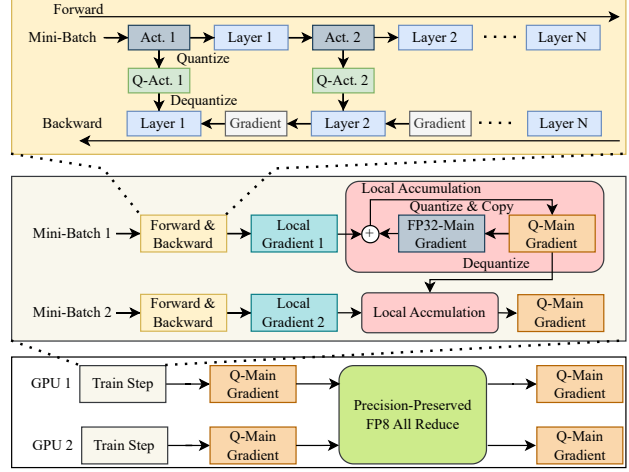


Figure 2. The integration of activation and gradient quantization with Megatron-LM.

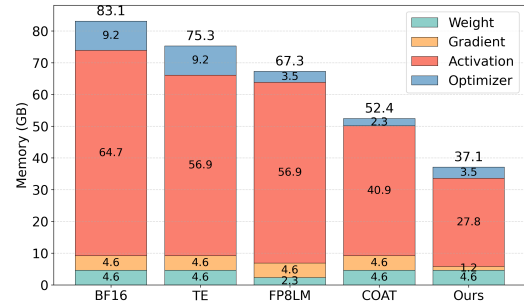


Figure 3. Training memory consumption on an OLMo-1B model.

tions after computation. When PP is employed, the amount of concurrently stored activations differs across devices, which will be discussed in Section 4.2.

3. AGoQ: System Overview

To significantly reduce the GPU memory footprint of LLM training, we design our AGoQ to compress activations to nearly 4 bits and gradients to 8 bits, which is also compatible with the 8-bit Adam optimizer (Dettmers et al., 2022) atop Megatron-LM. As shown in Fig. 2, AGoQ introduces two new components: nearly 4-bit activation quantization and 8-bit gradient quantization.

First, for activation quantization, the forward pass first generates full-precision activations, which are then quantized and stored in approximately 4-bit precision. During the backward pass, the quantized activations are dequantized to BF16/FP16 before gradient computation. In principle, 4-bit activations require only one quarter of the memory of FP16/BF16. However, naively quantizing the activations of all layers to 4 bits leads to substantial accuracy degradation. To gain the memory advantages of 4-bit while maintaining

model performance, we introduce layer-aware activation quantization (§4).

Second, for gradient quantization, at each GPU, the local gradient is first computed via forward and backward passes per mini-batch, then accumulated with the main gradient through local gradient accumulation. This process involves dequantizing the quantized main gradient (Q-Main Gradient), adding it to the local gradient in high precision, and quantizing the sum back to 8-bit format before copying it to the Q-Main Gradient. After local accumulation, GPUs perform a precision-preserved FP8 All-Reduce across GPUs, as described in §5.

As shown in Fig. 3 with an OLMo-1B model (Groeneveld et al., 2024), we compare the memory footprint across different components for the BF16 baseline, Transformer Engine (TE) (NVIDIA, 2024), FP8-LM (Peng et al., 2023c), COAT (Xi et al., 2025), and our method. AGoQ achieves further compression on both activations (§4) and gradients (§5): compared to COAT, we reduce activation memory by an additional 30% and gradient memory by 75%.

4. Layer-Aware Activation Quantization

To minimize accuracy degradation while maximizing overall memory savings, we first identify which layers’ activations are appropriate for 4-bit compression through a theoretical analysis, since different layer types (e.g., Attention, FFN, LayerNorm) exhibit distinct computation patterns as introduced in §2.1. Second, the PP training paradigm leads to uneven memory usage across different PP stages, which can be leveraged to design a dynamic quantization compensation strategy that takes advantage of underutilized memory resources.

4.1. Error Analysis of Activation Quantization

To minimize accuracy loss, we perform a numerical analysis to determine which activations should be quantized for different types of layers.

We first categorize different modules into two types based on whether they need to save additional activations beyond the input during computation. The matrix multiplication (GEMM) module in MLP only needs to save input activations. The modules that require saving additional activations include RMSNorm, SiLU & Multiply and attention modules. To illustrate the difference between the two types, we take RMSNorm (Eq. 4) as an example. Let

$$r = \sqrt{\frac{1}{d}\|X\|_2^2 + \epsilon}, \quad (6)$$

then it can be written as $Y = \text{RMSNorm}(X) = \gamma X/r$. The

gradient matrix is expressed as:

$$J = \frac{\text{diag}(\gamma)}{r} - \frac{1}{d} \frac{\text{diag}(\gamma)XX^T}{r^3}. \quad (7)$$

Therefore, to compute the gradient, we need to store both X and r . Here, r represents the additional activations that should also be cached. When using recomputation techniques, we do not store r ; instead, during the backward pass, r is recomputed from X before the gradient calculation.

For modules requiring additional activations, we primarily analyze gradient errors under two scenarios:

Case 1 (Recompute intermediate values): Only the quantized input activations are stored, and the originally required additional activations are recomputed during gradient calculation using the quantized input activations.

Case 2 (Cache intermediate values): Both the quantized input activations and the quantized additional activations are stored.

When analyzing GEMM computations adjacent to operations like RMSNorm or SiLU, we specifically compare two gradient computation strategies: one where only the quantized inputs to the preceding operation (e.g., RMSNorm/SiLU) are stored and the GEMM inputs are recomputed from these quantized values during backpropagation (also called Case 1), versus an alternative approach where the GEMM inputs themselves are directly stored in quantized form to avoid recomputation (Case 2).

In the following derivations, we make use of three standard norm inequalities. For vectors $X, Y \in \mathbb{R}^d$, the ℓ_2 -norm of their element-wise product satisfies

$$\|X \odot Y\|_2 \leq \|X\|_2 \|Y\|_\infty. \quad (8)$$

For matrices $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, the spectral norm is sub-multiplicative:

$$\|AB\|_2 \leq \|A\|_2 \|B\|_2 \quad (9)$$

$$\|AB\|_2 \leq \|A\|_2 \|B\|_\infty \quad (10)$$

4.1.1. RMSNORM

RMSNorm and its gradient are defined as Eq. 4 to Eq. 7.

Case 1 (Recompute intermediate values): Only the quantized input x is stored. We assume the error introduced by quantization can be modeled as a multiplicative perturbation:

$$X' = X \odot (1 + \delta X), \quad r' = r(X'), \quad J' = J(X'). \quad (11)$$

Let $\Delta J = J' - J$. Performing first-order expansion:

$$r' = r + \Delta r, \quad \Delta r \approx \frac{1}{2r} \frac{2}{d} \sum X_i^2 \delta X_i. \quad (12)$$

Table 1. Activation memory of different operations. U is a unit to measure memory usage, where 1U = Batch Size × Sequence Length × Hidden Size × 2 bytes (for BF16). Act Func refers to SiLU & Multiply.

	QKV	Attention	Linear	RMSNorm	FFN1	Act Func	FFN2	Total
Megatron-LM (w/ BF16)	1U	5U	1U	4U	1U	12U	4U	28U
COAT	1U	5U	1U	1U	0.5U	6U	2U	16.5U
AGoQ	0	5U	0.25U	0.5U	0	2U	0	7.75U

We obtain:

$$\Delta\left(\frac{1}{r}\right) \approx -\frac{\Delta r}{r^2}, \quad \Delta\left(\frac{1}{r^3}\right) \approx -3\frac{\Delta r}{r^4}. \quad (13)$$

Denoting $D_\gamma = \text{diag}(\gamma)$ and substituting into ΔJ , we have

$$\begin{aligned} \Delta J &\approx D_\gamma \left[\Delta\left(\frac{1}{r}\right) I - \frac{1}{d} \left(\Delta\left(\frac{1}{r^3}\right) X X^T + \frac{1}{r^3} \Delta(X X^T) \right) \right] \\ &= -\frac{\Delta r}{r^2} D_\gamma + \frac{3\Delta r}{dr^4} D_\gamma X X^T \\ &\quad - \frac{1}{dr^3} D_\gamma \left[X(\delta X \odot X)^T + (\delta X \odot X) X^T \right] \end{aligned}$$

Using Eq. 8 to Eq. 10, we have

$$\|\Delta J\|_2 \lesssim \|\gamma\|_\infty \left(\frac{|\Delta r|}{r^2} + \frac{3|\Delta r|}{dr^4} \|X\|_2^2 + \frac{2}{dr^3} \|X\|_2^2 \|\delta X\|_\infty \right), \quad (14)$$

where $\|\gamma\|_\infty = \max |\gamma_i|$. Further substituting Eq. 12, then we have

$$\|\Delta J\|_2 \lesssim 3\|\gamma\|_\infty \|\delta X\|_\infty \left(\frac{\|X\|_2^2}{dr^3} + \frac{\|X\|_2^4}{d^2 r^5} \right). \quad (15)$$

Typically ϵ is constant and $r \approx \sqrt{\|X\|_2^2/d}$, so the leading order is $\mathcal{O}(\|\gamma\|_\infty \|\delta X\|_\infty / r)$.

Case 2 (Cache intermediate values): Both the quantized input X and the quantized additional activation r are stored. In this case, the error is expressed as: $X' = X \odot (1 + \delta X)$ and $r' = r(1 + \delta r)$ be quantized, with $|\delta r|, \|\delta X\|_\infty \leq \epsilon_q$. The perturbed Jacobian is

$$J' = \frac{D_\gamma}{r'} - \frac{1}{d} \frac{D_\gamma X' (X')^T}{(r')^3}. \quad (16)$$

Expanding to first order:

$$\begin{aligned} \frac{1}{r'} &\approx \frac{1}{r} (1 - \delta r), \quad \frac{1}{(r')^3} \approx \frac{1}{r^3} (1 - 3\delta r), \\ X' (X')^T &\approx X X^T + X (X \odot \delta X)^T + (X \odot \delta X) X^T. \end{aligned} \quad (17)$$

Thus

$$\begin{aligned} J' &\approx \frac{D_\gamma}{r} - \frac{\delta r}{r} D_\gamma + \frac{3\delta r}{dr^3} D_\gamma X X^T \\ &\quad - \frac{1}{dr^3} D_\gamma \left[X X^T + X (X \odot \delta X)^T + (X \odot \delta X) X^T \right]. \end{aligned} \quad (18)$$

Subtracting the exact $J = \frac{D_\gamma}{r} - \frac{1}{dr^3} D_\gamma X X^T$ yields the first-order perturbation:

$$\begin{aligned} \Delta J &\approx -\frac{\delta r}{r} D_\gamma - \frac{1}{dr^3} D_\gamma \left[X (X \odot \delta X)^T + (X \odot \delta X) X^T \right] \\ &\quad + \frac{3\delta r}{dr^3} D_\gamma X X^T. \end{aligned} \quad (19)$$

Taking 2-norm bounds by Eq. 8 to Eq. 10 gives

$$\|\Delta J\|_2 \lesssim \frac{6\|\gamma\|_\infty \epsilon_q}{r}. \quad (20)$$

According to Eq. 15 and Eq. 20, we conclude that storing only the quantized input activations and recomputing intermediate values during gradient calculation yields the same asymptotic error order as caching intermediate values, with only constant-factor differences — and notably, the constant factor for the recomputation approach appears tighter. **Given that recompute also reduces memory footprint, for RMSNorm, we store the input activations as 4-bit and recompute their intermediate values for gradient computation.**

4.1.2. OTHER OPERATIONS

The detailed analyses of other operations like SiLU & Multiply, RMSNorm+GEMM, and Attention have similar derivations to RMSNorm, so they are provided in Appendix 8.1 and we conclude the following results.

We apply the Case 1 recomputation strategy to Q, K, V , intermediate activations of RMSNorm and SiLU & Multiply, input activations of two FFNs in MLP of each transformer layer, eliminating storage for those values. Based on the analysis in Appendix 8.1, the gradient error of quantizing activations of attention modules is significantly larger than that of other modules, so we do not quantize the activations of attention modules. Other activations are quantized to 4 bits via block-wise FP4 quantization (Dettmers et al., 2022; Li et al., 2023) with the blocksize of 128. As shown in Table 1, our approach reduces RMSNorm memory from 4U¹ to 0.5U (4-bit), and SiLU & Multiply activations from 12U to 2U. Attention remains at 5U as reported in (Xi et al., 2025). Overall, activation memory drops from 28U in

¹Note that in Megatron-LM with BF16 training, RMSNorm still uses FP32 for better convergence.

Megatron-LM to 7.75U in our method—an approximate three-fold reduction.

4.2. Dynamic Bit-width Compensation

Interleaved 1F1B PP leads to imbalanced memory footprints across devices. As illustrated in Fig. 1 (four PP stages with 8 mini-batches), different devices store varying numbers of activation batches—e.g., Device 1 holds 11 mini-batch activations at peak, while Devices 2, 3, and 4 store only 9, 7, and 5, respectively. This results in significant under-utilization of GPU memory, with Device 1 occupying $2.2\times$ more activation memory than Device 4.

To exploit the under-utilized memory, we propose Dynamic Bit-width Compensation for Activation with Pipeline Parallelism (DBCA-PP). Devices storing fewer activation batches are assigned higher quantization bit-widths, thereby compensating for quantization-induced precision loss without increasing peak memory usage. This strategy makes full use of the otherwise wasted memory across the pipeline, while keeping nearly 4-bit activation storage.

Formally, in Interleaved 1F1B PP, the number of activation mini-batches N_i stored at stage i (with totally n stages) can be expressed as:

$$N_i = n + 2 \cdot i - 1, \quad 1 \leq i \leq n. \quad (21)$$

Based on the memory availability per stage, the quantization bit-width B_i (with a minimum of 4) for activations at stage i can be set as inversely proportional to the number of activation mini-batches N_i :

$$B_i = \frac{4 \cdot N_1}{N_i}, \quad 1 \leq i \leq n. \quad (22)$$

It is worth mentioning that the bit-width configuration generated for a lower number of stages can also be directly applied to a setup with a higher number of stages. In other words, directly reusing the bit assignment scheme from a lower-stage configuration in a higher-stage setup with DBCA-PP would not increase the peak GPU memory usage of the higher-stage setup.

4.3. Kernel Fusion of Quantization/Dequantization and GEMM

The activation quantization and dequantization require extra computation overheads. To address this problem, we fuse these operations along with nearby GEMM computations into a single GPU kernel. This is motivated by the fact that quantization and dequantization are mainly element-wise operations, thus only utilize CUDA cores, whereas GEMM leverages Tensor Cores on modern GPUs.

To achieve this goal, we carefully schedule the execution of activation quantization, dequantization, and GEMM operations during LLM training as shown in Fig. 4. During

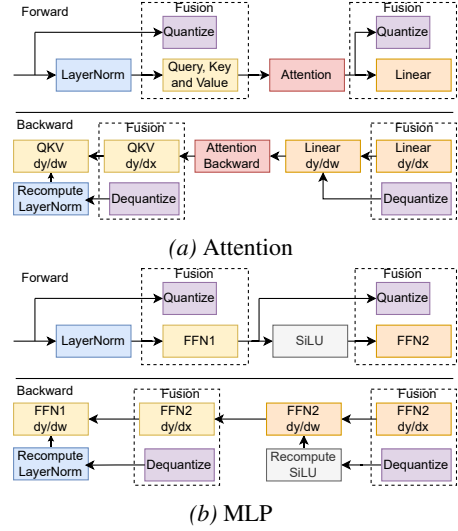


Figure 4. The forward and backward passes of attention and MLP for kernel fusion of quantization/dequantization and GEMM.

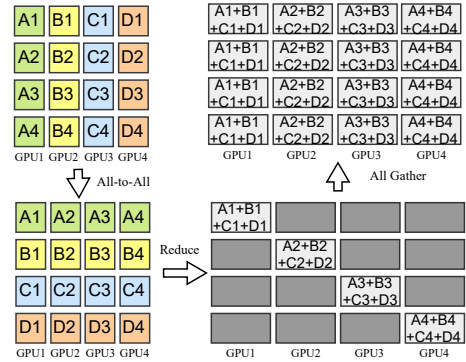


Figure 5. The illustration of our process to perform All-Reduce by combining All-to-All with All-Gather.

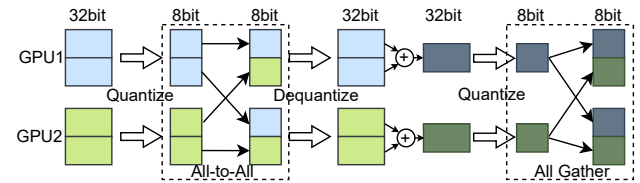


Figure 6. The illustration of our process to combine gradients from different GPUs.

the forward pass, we fuse the quantization process with its subsequent GEMM operation. During the backward pass, we fuse dequantization with the GEMM operation that is responsible for computing activation gradients. This approach can almost eliminate the computation overheads of activation quantization, thus improving execution efficiency.

5. Precision-Preserved Gradient Quantization

To minimize both the memory usage associated with storing gradients and the communication overhead during gradient All-Reduce, we introduce an 8-bit block-wise (Dettmers et al., 2022) gradient quantization technique. This method maintains precision throughout the All-Reduce operation and effectively mitigates two distinct overflow issues found within the accumulation process.

First, in LLM training, a global batch is typically divided into multiple mini-batches whose gradients are locally accumulated before communication with other DP workers. When we store the main gradients with FP8, directly accumulating gradients from different mini-batches would easily cause overflow. Thus, in local gradient accumulation, we dequantize the FP8 main gradient to FP16/BF16 for high-precision addition of different mini-batch gradients, and the final results are then quantized to FP8 as shown in Fig. 2.

Second, the gradients should be aggregated among DP workers via an All-Reduce operation, which can be divided into Reduce-Scatter and All-Gather. However, Reduce-Scatter requires performing addition during communication, which could easily cause an overflow in FP8. Thus, we split the All-Reduce operation into an All-to-All operation with a local reduce followed by an All-Gather operation as shown in Fig. 5. As shown in Fig. 6, the FP8 gradients are communicated via an All-to-All communication to send the compressed data to all devices. Each device then dequantizes the data received from different devices to FP32, performs local reduce operations, and then quantizes the result again for its following All-Gather operation. After that, we perform All-Gather on the summed data to complete the All-Reduce operation.

6. Evaluation

6.1. Experimental Settings

Testbeds. Experiments are mainly carried out on a 64-GPU cluster connected with 200Gb/s InfiniBand comprising 8 nodes, each of which is equipped with eight Nvidia A6000 GPUs. In the comparative experiments with COAT (Xi et al., 2025), we used two nodes, each equipped with 8 NVIDIA Pro 6000 GPUs, to support the FP8 format required by COAT. The software environments are Ubuntu-20.04, CUDA-12.1, PyTorch-2.1.2, and NCCL-2.18.5. Our system also supports Huawei Ascend 910 NPU (more results can be found in Appendix 8.2).

Baselines. We implement our AGoQ atop Megatron-LM. We compare our AGoQ with three representative baselines Megatron-LM (Narayanan et al., 2021) (w/o and w/ ZeRO (Rajbhandari et al., 2020)), DeepSpeed (Rasley et al., 2020), and COAT (Xi et al., 2025).

Models. We primarily conduct pre-training experiments to verify convergence on LLaMA2-7B (Touvron et al., 2023) due to extremely high training costs, and perform training time comparison experiments on larger models including LLaMA3-8B (Dubey et al., 2024), LLaMA2-13B (Touvron et al., 2023), and CodeLLaMA-34B (Roziere et al., 2023). When comparing with COAT, we employed the OLMo-1B model (Groeneveld et al., 2024) provided by the original COAT paper (Xi et al., 2025).

Table 2. Performance Comparison of AGoQ, Megatron-LM and ZeRO-1 on LLaMA2-13B. R means the number of recomputed transformer layers. The unit of time is milliseconds (ms).

Sequence Length	Megatron-LM		ZeRO-1		AGoQ	
	R	Time	R	Time	R	Time
32K	3	37635	2	37038	0	36568
40K	4	51922	4	51418	0	45590
48K	6	67932	6	67928	0	57047
56K	8	88200	8	86601	0	69544
64K	8	104444	8	103706	0	82519
72K	10	128085	10	128152	0	97615
80K	10	149667	10	149288	0	111422

6.2. End-to-end Training Time Comparison

LLaMA2-13B. To assess the effectiveness of our method, we compare AGoQ with Megatron-LM without ZeRO-1 and with ZeRO-1. We benchmark training speed on LLaMA2-13B with sequence lengths from 32K to 80K. Under limited GPU memory, we adopt selective activation recomputation: instead of caching all intermediate activations after the forward pass, we recompute them via an extra forward pass during backpropagation. The number of transformer layers that use recomputation is adaptively tuned based on real-time memory usage. Table 2 reports results on 64 GPUs with mini-batch size 1, global batch size 16, PP=4 and TP=8. Overall, AGoQ achieves an average speedup of $1.22\times$ over Megatron-LM and $1.21\times$ over its ZeRO-1 variant (we focus on ZeRO-1 here because ZeRO-2 and ZeRO-3 introduce additional communication overhead, typically reducing throughput, as shown in §8.2 in Appendix). The speedup grows with sequence length, confirming the benefit of our approach; for instance, at 80K tokens, AGoQ is about $1.34\times$ faster than both Megatron-LM and ZeRO-1.

Performance under Different Configurations. To further evaluate our method, we broaden the experimental settings. We vary the sequence length from 16K to 32K or from 32K to 80K, set pipeline parallelism (PP) from 1 up to the number of nodes, tensor parallelism (TP) in 4, 8, and the number of GPUs in 8, 16, 32, 64. We test three models: LLaMA3-8B, LLaMA2-13B, and CodeLLaMA-34B. The detailed configurations are summarized in Appendix 8.2, where we run 124 experiments each for Megatron-LM, ZeRO-1, and AGoQ, for a total of 372 experiments. Of these, 69 runs failed due

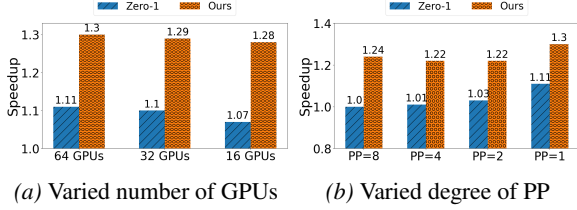


Figure 7. Speedups of our AGoQ and ZeRO-1 over Megatron-LM on varied configurations.

Table 3. Time and memory at different sequence lengths.

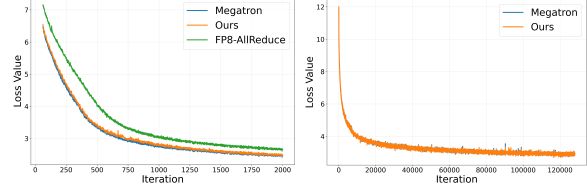
Seq. Len.	Method	Time (ms)	Memory (MB)
24k	COAT	6291	94100
	AGoQ	6161	66852
32k	COAT	8861	95664
	AGoQ	8076	86012

to out-of-memory (OOM) errors (AGoQ, Megatron-LM, and ZeRO-1 have 16, 33, and 20 OOM cases, respectively), and 303 runs completed successfully. Overall, the results show that AGoQ delivers an average speedup of $1.23\times$ over Megatron-LM and $1.19\times$ over ZeRO-1. We next examine the impact of GPU count, sequence length, model size, and PP separately. Results are shown in Fig. 7, which indicates that AGoQ consistently achieves substantial improvements over Zero-1 and Megatron-LM across different GPU counts and degree of PP.

Comparison with COAT. We further compare AGoQ with COAT, using two nodes with 8 Pro6000 GPUs to support COAT’s FP8 format. With a global batch size of 64 and sequence lengths of 24K and 32K, we evaluate the OLMo-1B model. For 32K sequences, COAT encounters OOM errors, requiring recomputation for half of the transformer layers. Results in Table 3 show that at 24K, our AGoQ reduces memory by 31% over COAT while matching training speed; at 32K, with recomputation enabled for COAT, AGoQ achieves a $1.1\times$ end-to-end speedup. Due to our hardware limit, we only conduct the experiments on 16 Blackwell GPUs. It is expected to achieve higher speedups over COAT on larger clusters since AGoQ allows 8-bit communication to significantly reduce DP communication overhead.

6.3. Convergence Loss

We evaluate the convergence of AGoQ by pretraining LLaMA2-7B and LLaMA3-8B on 2B tokens from Open-WebText (Gokaslan & Cohen, 2019). We evaluate the robustness of our method by testing it with different global batch sizes on two models: 512 in LLaMA2-7B and 4 in LLaMA3-8B. Using an interleaved 1F1B schedule with 4 pipeline stages, we apply DBCA-PP with activation bit-widths of 4,



(a) Llama2-7B

(b) Llama3-8B

Figure 8. Training loss of Megatron (w/ BF16), FP8-AllReduce and ours (A+O+G) on LLaMA2-7B and LLaMA3-8B.

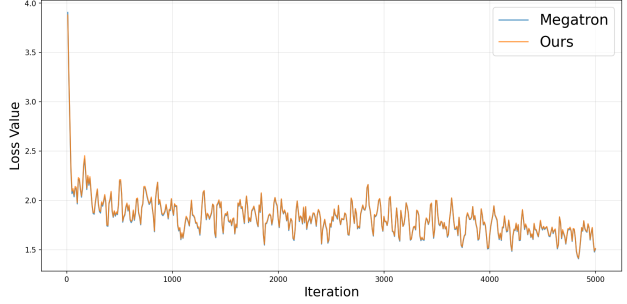


Figure 9. SFT training loss of Megatron (w/ BF16) and ours on VLM tasks.

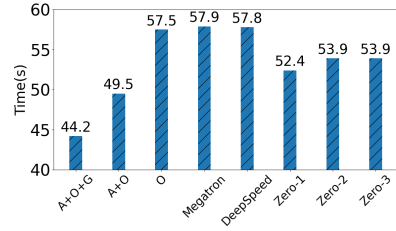


Figure 10. Iteration time (in seconds) comparison on LLaMA2-13B.

5, 6, and 8 per stage according to Eq. 22, matching the peak memory footprint of uniform 4-bit compression. During the training of LLaMA2-7B, we additionally tested the training curve of FP8-AllReduce (Microsoft’s FP8 AllReduce method (Peng et al., 2023b)). As shown in Fig. 8, our approach closely track the baseline loss, while FP8-AllReduce shows significantly higher loss.

We also compared the training curves of LLM models trained by AGoQ and Megatron on downstream VLM tasks, using the small-size Supervised Fine-Tuning (SFT) training set provided in Open-Qwen2VL (Wang et al., 2025). The results show that the losses almost overlap.

Comparison of Different Optimizations. We also validated the individual contributions of activation quantization, gradient quantization, and optimizer quantization modules, while simultaneously examining the differences between DeepSpeed, ZeRO-1, ZeRO-2, and ZeRO-3. Under the configuration of PP=1 and sequence length=48K, we conducted

Table 4. Communication latency breakdown (ms) under 200Gbps and 10Gbps bandwidth.

Message Size	All-Reduce	All-to-All	Quant/Dequant	All-Gather	AGoQ
2^{30} (1GB)	4292.77 / 50365.45	599.28 / 7297.73	31.03 / 31.07	556.47 / 226.58	1186.78 / 7555.38
2^{25} (32MB)	131.23 / 1603.78	18.81 / 233.34	0.99 / 1.10	19.37 / 197.92	39.17 / 432.36
2^{20} (1MB)	4.13 / 51.07	0.83 / 5.52	0.07 / 0.07	0.55 / 7.87	1.45 / 13.46
2^{15} (32KB)	0.83 / 3.93	0.35 / 0.38	0.05 / 0.05	0.41 / 0.83	0.81 / 1.26

Table 5. Baseline wall-clock time breakdown (ms) per Transformer decoder layer.

Phase	ln	ag/rs	Attn	rs/ag	ln	ag/rs	FFN	rs/ag
Forward	1.3	17.02	19.34	21.79	1.3	17	13.3	23.01
Backward	2.85	23.21	45.06	35.4	3.2	22.62	28.6	34

experiments on LLaMA2-13B with three different settings: “A+O+G” (i.e., AGoQ), “A+O” (only applying activation and optimizer quantization) and “O” (only applying 8-bit optimizer quantization) shown in Fig. 10, demonstrate that iteration time progressively decreases as additional quantization modules are incorporated. It also indicates that the training speed decreases from ZeRO-1 to ZeRO-2 and ZeRO-3, and our AGoQ significantly outperforms Megatron-LM, DeepSpeed, and ZeRO series.

Communication Savings on Commodity Bandwidth (e.g., 100Gbps). We evaluated communication efficiency under two representative bandwidth conditions: 200Gbps (our primary testbed) and 10Gbps (to simulate commodity constraints). Table 4 reports latency breakdowns for both configurations (200Gbps / 10Gbps) under TP=8, DP=8. At 32MB under 200Gbps, our decomposed approach achieves a $3.4\times$ speedup (39.17 ms vs. 131.23 ms). Under 10Gbps, the speedup increases to $3.7\times$ (432.36 ms vs. 1603.78 ms), confirming that our method delivers substantial communication savings across diverse bandwidth settings.

Wall-Clock Time Breakdown. We performed a detailed timing breakdown for a single Transformer decoder layer (ms). Due to Megatron’s sequence parallelism (SP), All-Gather costs are doubled during backward pass. Quantization/dequantization are fused into GEMM kernels. Table 5 and Table 6 report the breakdowns for the baseline and AGoQ, respectively. The results show that AGoQ introduces minimal overhead in compute-bound operations while effectively saving memory. The fused quantization/dequantization adds negligible latency, as evidenced by the small increase in FFN forward time (13.3 $\rightarrow$ 15.64 ms), FFN backward time (28.6 $\rightarrow$ 32.54 ms), Attn forward time (19.3 $\rightarrow$ 20.6 ms) and Attn backward time (45.1 $\rightarrow$ 47.8 ms).

Extended Throughput Analysis (2k/4k/8k). We extended our throughput analysis to sequence lengths of 2k, 4k, and

Table 6. AGoQ wall-clock time breakdown (ms) per Transformer decoder layer.

Phase	ln	ag/rs	Attn	rs/ag	ln	ag/rs	FFN	rs/ag
Forward	1.3	16.82	20.62	21.65	1.3	17	15.64	21.78
Backward	2.9	22.82	47.86	34.56	3.4	22.58	32.54	33.54

Table 7. Throughput (samples/sec) comparison at sequence lengths 2k, 4k, and 8k.

Seq.	Megatron-LM	ZeRO-1	AGoQ (Ours)
2k	2862.22	2498.13	2148.82
4k	4626.94	4261.65	3968.20
8k	8188.41	7848.12	7755.74

8k. When memory is sufficient and recomputation is not required, we recommend enabling only gradient compression, as it effectively reduces communication overhead. Table 7 shows the throughput (samples/sec) comparison with Megatron-LM and ZeRO-1 baselines. Our method consistently outperforms both baselines across all sequence lengths, achieving speedups of up to $1.33\times$ over Megatron-LM and $1.16\times$ over ZeRO-1 at 2k sequence length.

In the Appendix 8.2, we also present several ablation studies.

7. Conclusion

In this work, we addressed the critical challenge of GPU memory consumption in LLM training through a holistic quantization approach. Specifically, we present AGoQ, which integrates: 1) a layer-aware activation quantization strategy that assigns suitable bits for storing activations based on layer types and pipeline parallelism stages, and 2) a gradient quantization algorithm that conserves memory and reduces communication time by using low-bit gradient storage and precision-preserved low-bit data All-Reduce communication. Extensive experiments on two GPU clusters (up to 64 GPUs) demonstrate that AGoQ reduces memory usage by 52% compared to full-precision training and improves end-to-end training throughput up to $1.34\times$ over state-of-the-art systems including Megatron-LM, DeepSpeed, ZeRO and COAT, while maintaining competitive accuracy on downstream tasks with LLaMA architectures.

Impact Statement

This paper proposes AGoQ, a memory-efficient distributed training system for large language models (LLMs) based on layer-aware activation quantization and precision-preserved gradient quantization. By reducing activation storage to near 4-bit precision and gradient storage/communication to 8-bit precision, AGoQ aims to reduce GPU memory consumption and improve training efficiency. The potential positive impact of this work is that it may lower the hardware and financial barriers for training LLMs, particularly for research groups and organizations with limited GPU memory or communication bandwidth. It may also improve the energy and resource efficiency of LLM training by reducing memory pressure, recomputation overhead, and communication cost.

Acknowledgments

This work was partially supported by the Shenzhen Science and Technology Program under Grant No. KJZD20240903104103005.

References

- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Bai, Y., Li, C., Zhou, Q., Yi, J., Gong, P., Yan, F., Chen, R., and Xu, Y. Gradient compression supercharged high-performance data parallel dnn training. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pp. 359–375, 2021.
- Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chen, P., Deng, Z., Li, P., He, S., Zhu, H., Zheng, Y., Wang, Z., Huai, B., and Guo, M. Adacc: An adaptive framework unifying compression and activation recomputation for llm training. *arXiv preprint arXiv:2508.00806*, 2025.
- Chen, T., Xu, B., Zhang, C., and Guestrin, C. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Dean, J., Corrado, G., Monga, R., Chen, K., Devin, M., Mao, M., Ranzato, M., Senior, A., Tucker, P., Yang, K., et al. Large scale distributed deep networks. *Advances in neural information processing systems*, 25, 2012.
- Dettmers, T., Lewis, M., Shleifer, S., and Zettlemoyer, L. 8-bit optimizers via block-wise quantization. In *International Conference on Learning Representations*, 2022.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv e-prints*, pp. arXiv–2407, 2024.
- Evans, R. D. and Aamodt, T. Ac-gc: Lossy activation compression with guaranteed convergence. *Advances in Neural Information Processing Systems*, 34:27434–27448, 2021.
- Gokaslan, A. and Cohen, V. Openwebtext corpus. <http://Skyllion007.github.io/OpenWebTextCorpus>, 2019.
- Groeneveld, D., Beltagy, I., Walsh, P., Bhagia, A., Kinney, R., Tafjord, O., Jha, A., Ivison, H., Magnusson, I., Wang, Y., Arora, S., Atkinson, D., Authur, R., Chandu, K. R., Cohan, A., Dumas, J., Elazar, Y., Gu, Y., Hessel, J., Khot, T., Merrill, W., Morrison, J. D., Muennighoff, N., Naik, A., Nam, C., Peters, M. E., Pyatkin, V., Ravichander, A., Schwenk, D., Shah, S., Smith, W., Strubell, E., Subramani, N., Wortsman, M., Dasigi, P., Lambert, N., Richardson, K., Zettlemoyer, L., Dodge, J., Lo, K., Soldaini, L., Smith, N. A., and Hajishirzi, H. OLMo: Accelerating the science of language models. *arXiv preprint*, 2024. URL <https://api.semanticscholar.org/CorpusID:267365485>.
- Huang, J., Di, S., Yu, X., Zhai, Y., Liu, J., Huang, Y., Raffanetti, K., Zhou, H., Zhao, K., Lu, X., et al. gzcl: Compression-accelerated collective communication framework for gpu clusters. In *Proceedings of the 38th ACM International Conference on Supercomputing*, pp. 437–448, 2024.
- Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- Li, B., Chen, J., and Zhu, J. Memory efficient optimizers with 4-bit states. *Advances in Neural Information Processing Systems*, 36:15136–15171, 2023.
- Lin, W., Pan, X., Shi, S., Wang, X., Li, B., and Chu, X. Mast: Efficient training of mixture-of-experts transformers with

- task pipelining and ordering. In *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*, pp. 560–570. IEEE, 2025.
- Liu, X., Zheng, L., Wang, D., Cen, Y., Chen, W., Han, X., Chen, J., Liu, Z., Tang, J., Gonzalez, J., et al. Gact: Activation compressed training for generic network architectures. In *International Conference on Machine Learning*, pp. 14139–14152. PMLR, 2022.
- Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., et al. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, 2021.
- NVIDIA. Transformer engine: An efficient library for training transformer models, 2024. URL <https://github.com/NVIDIA/TransformerEngine>. Accessed: 2024-09-19.
- Pan, X., Lin, W., Shi, S., Chu, X., Sun, W., and Li, B. Parm: Efficient training of large sparsely-activated models with dedicated schedules. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, 2024.
- Pan, X., Lin, W., Zhang, L., Shi, S., Tang, Z., Wang, R., Li, B., and Chu, X. FSMoE: A flexible and scalable training system for sparse mixture-of-experts models. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pp. 524–539, 2025.
- Peng, H., Qin, S., Yu, Y., Wang, J., Wang, H., and Li, G. Birder: communication-efficient 1-bit adaptive optimizer for practical distributed dnn training. *Advances in Neural Information Processing Systems*, 36:39529–39552, 2023a.
- Peng, H., Wu, K., Wei, Y., Zhao, G., Yang, Y., Liu, Z., Xiong, Y., Yang, Z., Ni, B., Hu, J., Li, R., Zhang, M., Li, C., Ning, J., Wang, R., Zhang, Z., Liu, S., Chau, J., Hu, H., and Cheng, P. FP8-LM: training FP8 large language models. *CoRR*, abs/2310.18313, 2023b.
- Peng, H., Wu, K., Wei, Y., Zhao, G., Yang, Y., Liu, Z., Xiong, Y., Yang, Z., Ni, B., Hu, J., et al. Fp8-lm: Training fp8 large language models. *arXiv preprint arXiv:2310.18313*, 2023c.
- Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–16. IEEE, 2020.
- Rajbhandari, S., Ruwase, O., Rasley, J., Smith, S., and He, Y. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pp. 1–14, 2021.
- Ramachandran, P., Zoph, B., and Le, Q. V. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.
- Rasley, J., Rajbhandari, S., Ruwase, O., and He, Y. Deep-speed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 3505–3506, 2020.
- Ren, J., Rajbhandari, S., Aminabadi, R. Y., Ruwase, O., Yang, S., Zhang, M., Li, D., and He, Y. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pp. 551–564, 2021.
- Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Shamshoum, Y., Hodos, N., Sieradzki, Y., and Schuster, A. CompAct: Compressed activations for memory-efficient llm training. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 1511–1524, 2025.
- Shi, S., Zhou, X., Song, S., Wang, X., Zhu, Z., Huang, X., Jiang, X., Zhou, F., Guo, Z., Xie, L., et al. Towards scalable distributed training of deep learning on public cloud clusters. *Proceedings of Machine Learning and Systems*, 3:401–412, 2021.
- Shi, S., Pan, X., Wang, Q., Liu, C., Ren, X., Hu, Z., Yang, Y., Li, B., and Chu, X. ScheMoE: An extensible mixture-of-experts distributed training system with tasks scheduling. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pp. 236–249, 2024.
- Tang, H., Gan, S., Awan, A. A., Rajbhandari, S., Li, C., Lian, X., Liu, J., Zhang, C., and He, Y. 1-bit adam: Communication efficient large-scale training with adam’s convergence speed. In *International Conference on Machine Learning*, pp. 10118–10129. PMLR, 2021.

- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wang, G., Qin, H., Jacobs, S. A., Wu, X., Holmes, C., Yao, Z., Rajbhandari, S., Ruwase, O., Yan, F., Yang, L., and He, Y. ZeRO++: Extremely efficient collective communication for large model training. In *The Twelfth International Conference on Learning Representations*, 2024.
- Wang, W., Tian, Y., Yang, L., Wang, H., and Yan, X. OpenQwen2VL: Compute-efficient pre-training of fully-open multimodal llms on academic resources. *arXiv preprint arXiv:2504.00595*, 2025.
- Welbl, J., Liu, N. F., and Gardner, M. Crowdsourcing multiple choice science questions. *arXiv preprint arXiv:1707.06209*, 2017.
- Wu, K., Park, J. B., Zhang, X., Hidayetoğlu, M., Maitlthody, V. S., Huang, S., Lumetta, S., and Hwu, W.-M. Ssdtrain: An activation offloading framework to ssds for faster large language model training. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pp. 1–7. IEEE, 2025.
- Xi, H., Chen, Y., Zhao, K., TEH, K. J., Chen, J., and Zhu, J. Jetfire: efficient and accurate transformer pretraining with int8 data flow and per-block quantization. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 54049–54063, 2024.
- Xi, H., Cai, H., Zhu, L., Lu, Y., Keutzer, K., Chen, J., and Han, S. COAT: Compressing optimizer states and activations for memory-efficient FP8 training. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=XfKSDgqIRj>.
- Yuan, T., Liu, Y., Ye, X., Zhang, S., Tan, J., Chen, B., Song, C., and Zhang, D. Accelerating the training of large language models using efficient activation rematerialization and optimal hybrid parallelism. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pp. 545–561, 2024.
- Zhang, B. and Sennrich, R. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.
- Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., et al. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12): 3848–3860, 2023.

8. Appendix

8.1. Error Analysis of Activation Quantization

We conduct an error analysis for SiLU & Multiply, RM-SNorm+GEMM, and Attention mentioned in § 4.1.

8.1.1. SiLU & MULTIPLY

SiLU & Multiply is an element-wise operation that takes two inputs, denoted as X and Y . For a pair of corresponding elements x and y from these inputs, the SiLU & Multiply operation is defined as:

$$z = xy\sigma(y)$$

where σ is the sigmoid function. Its derivatives are

$$\frac{\partial z}{\partial x} = y\sigma(y), \quad \frac{\partial z}{\partial y} = x\sigma(y) + xy\sigma(y)(1 - \sigma(y)),$$

and perturbations are

$$x' = x(1 + \delta x), \quad y' = y(1 + \delta y).$$

Case 1 (Recompute intermediate values)

$$\begin{aligned} \Delta \frac{\partial z}{\partial x} &\approx (\sigma(y) + y\sigma'(y))y\delta y. \\ |\Delta \frac{\partial z}{\partial x}| &\leq |y|(\sigma(y) + |y|\sigma(y)(1 - \sigma(y)))|\delta y| \\ &\leq \mathcal{O}(|y|^2|\delta y|). \end{aligned}$$

For $\partial z/\partial y$:

$$\begin{aligned} \Delta \frac{\partial z}{\partial y} &\approx x\sigma(y)(1 + y(1 - \sigma(y)))\delta x \\ &\quad + x\sigma(y)(1 - \sigma(y))(2 + y(1 - 2\sigma(y)))y\delta y. \\ |\Delta \frac{\partial z}{\partial y}| &\leq |x|\sigma(y)|1 + y(1 - \sigma(y))||\delta x| \\ &\quad + |x|\sigma(y)(1 - \sigma(y))|y||2 + y(1 - 2\sigma(y))||\delta y|. \end{aligned}$$

Asymptotically: $\mathcal{O}(|x||\delta x| + |x||y||\delta y|)$.

Case 2 (Cache intermediate values)

$$\sigma' \approx \sigma(y)(1 + \delta s), \quad \frac{\partial z'}{\partial x'} = y'\sigma' \approx y(1 + \delta y)\sigma(y)(1 + \delta s).$$

$$\begin{aligned} \Delta \frac{\partial z}{\partial x} &\approx y\sigma(y)(\delta y + \delta s). \\ |\Delta \frac{\partial z}{\partial x}| &\leq |y|\sigma(y)(|\delta y| + |\delta s|) \leq \mathcal{O}(|y|(|\delta y| + |\delta s|)). \end{aligned}$$

For $\frac{\partial z'}{\partial y'}$:

$$\frac{\partial z'}{\partial y'} = x'\sigma' + x'y'\sigma'(1 - \sigma').$$

Expanding to first order:

$$\begin{aligned} \frac{\partial z'}{\partial y'} &\approx x\sigma(y)[1 + \delta x + \delta s] \\ &\quad + xy\sigma(y)(1 - \sigma(y))[1 + \delta x + \delta y + \delta s - \frac{\sigma(y)}{1 - \sigma(y)}\delta s]. \end{aligned}$$

Subtracting the exact $\frac{\partial z}{\partial y} = x\sigma(y) + xy\sigma(y)(1 - \sigma(y))$:

$$\begin{aligned} \Delta \frac{\partial z}{\partial y} &\approx x\sigma(y)[1 + y(1 - \sigma(y))]\delta x \\ &\quad + xy\sigma(y)(1 - \sigma(y))\delta y \\ &\quad + x\sigma(y)[1 + y(1 - 2\sigma(y))]\delta s. \end{aligned}$$

$$\begin{aligned} \left| \Delta \frac{\partial z}{\partial y} \right| &\leq |x|\sigma(y)|1 + y(1 - \sigma(y))||\delta x| \\ &\quad + |x||y|\sigma(y)(1 - \sigma(y))|\delta y| \\ &\quad + |x|\sigma(y)|1 + y(1 - 2\sigma(y))||\delta s|. \end{aligned}$$

Asymptotically:

$$|\Delta \frac{\partial z}{\partial y}| = \mathcal{O}(|x|(|\delta x| + |\delta y| + |y||\delta s|)).$$

Comparing the two cases for the SiLU & Multiply operation, under the common scenario where the inputs x and y are mostly smaller than 1, ****Case 1 (recomputing intermediate values) gives a strictly smaller error upper bound**** than Case 2 (caching intermediate values). For $\partial z/\partial x$, Case 1 yields $\mathcal{O}(|y|^2|\delta y|)$ while Case 2 yields $\mathcal{O}(|y|(|\delta y| + |\delta s|))$; because $|y| \leq 1$ implies $|y|^2|\delta y| \leq |y||\delta y| \leq |y|(|\delta y| + |\delta s|)$, the recompute bound is always tighter. For $\partial z/\partial y$, Case 1's asymptotic bound $\mathcal{O}(|x||\delta x| + |x||y||\delta y|)$ is also lower than Case 2's $\mathcal{O}(|x|(|\delta x| + |\delta y| + |y||\delta s|))$, since the latter contains an extra term proportional to $|\delta s|$ that is absent when the sigmoid is recomputed exactly from the perturbed input. Thus, as long as the cache error δs is non-negligible and the typical input magnitudes satisfy $|x|, |y| < 1$, recomputing the intermediate values on the fly produces a provably smaller worst-case gradient error. **Therefore, for SiLU computation, we only store the quantized input activations and recompute the necessary intermediate values during the backward pass.**

8.1.2. RMSNorm + GEMM

Consider $Y = WU$, where $U = \text{RMSNorm}(X)$. The gradient w.r.t. W is

$$\frac{\partial Y}{\partial W} = U^T, \quad U = \gamma X/r.$$

Case 1 (Recompute intermediate values)

Input perturbation:

$$X' = X \odot (1 + \delta X), \quad r' = r(X'), \quad U' = \gamma X'/r'.$$

Gradient error:

$$\Delta \left(\frac{\partial Y}{\partial W} \right) = U'^T - U^T.$$

First-order expansion:

$$\|U' - U\|_2 \leq \|\gamma\|_\infty \left(\frac{\|X\|_2}{r^2} + \frac{\|X\|_2^3}{dr^4} \right) \|\delta X\|_\infty.$$

Asymptotically: $\mathcal{O}(d\|\gamma\|_\infty\|\delta X\|_\infty/\|X\|_2)$.

Case 2 (Cache intermediate values)

$$U_c = U \odot (1 + \delta U).$$

Gradient error bound:

$$\|\Delta U\|_2 = \|U \odot \delta U\|_2 \leq \|U\|_2 \|\delta U\|_\infty.$$

Asymptotically: $\mathcal{O}(\|U\|_2 \|\delta U\|_\infty)$.

Two cases yield the same asymptotic error order, with only constant-factor differences. **Given that recompute also reduces memory footprint, for GEMM computation, we only keep the quantized input activations of RMSNorm.** Similarly, it was found that storing only the input activations of SiLU and recalculating the input of GEMM during gradient computation for GEMM after SiLU also decreases the upper bound on gradient error.

8.1.3. ATTENTION

A single-head attention is:

$$A = PV = \text{softmax}(S)V, \quad S = \frac{QK^T}{\sqrt{d}}.$$

Its gradients are

$$\begin{aligned} \frac{\partial A}{\partial V} &= P^\top, & \frac{\partial A}{\partial P} &= V^\top, \\ \frac{\partial A}{\partial S} &= P \left(\frac{\partial A}{\partial P} - z \mathbf{1}^\top \right), & z &= \text{rowsum} \left(P \frac{\partial A}{\partial P} \right), \\ \frac{\partial A}{\partial Q} &= \frac{1}{\sqrt{d}} \frac{\partial A}{\partial S} K, & \frac{\partial A}{\partial K} &= \frac{1}{\sqrt{d}} \left(\frac{\partial A}{\partial S} \right)^\top Q, \end{aligned}$$

and perturbations are

$$Q' = Q \odot (1 + \delta Q), \quad K' = K \odot (1 + \delta K), \quad V' = V \odot (1 + \delta V).$$

Case 1 (Recompute intermediate values).

$$A' = \text{attention}(Q', K', V')$$

Thus, the gradient errors can be represented as:

$$\begin{aligned} \|\Delta A\|_2 &\leq \|\Delta S\|_2 \|V\|_2 + \|V \odot \delta V\|_2 \\ \|\Delta \frac{\partial A}{\partial V}\|_2 &\leq \frac{1}{\sqrt{d}} \|Q \odot \delta Q K^\top + Q(K \odot \delta K)^\top\|_2 \\ &= \mathcal{O} \left(\frac{\|Q\|_2 \|K\|_2}{\sqrt{d}} (\|\delta Q\|_\infty + \|\delta K\|_\infty) \right). \end{aligned}$$

$$\begin{aligned} \|\Delta \frac{\partial A}{\partial K}\|_2 &\leq \frac{\|V\|_2 \|K\|_2}{d} \|Q \odot \delta Q K^\top + Q(K \odot \delta K)^\top\|_2 \\ &= \mathcal{O} \left(\frac{\|Q\|_2 \|K\|_2 \|V\|_2}{d} (\|\delta Q\|_\infty + \|\delta K\|_\infty) \right). \end{aligned}$$

$$\begin{aligned} \|\Delta \frac{\partial A}{\partial Q}\|_2 &\leq \frac{\|V\|_2 \|Q\|_2}{d} \|Q \odot \delta Q K^\top + Q(K \odot \delta K)^\top\|_2 \\ &= \mathcal{O} \left(\frac{\|V\|_2 \|Q\|_2 \|K\|_2}{d} (\|\delta Q\|_\infty + \|\delta K\|_\infty) \right). \end{aligned}$$

Case 2 (Cache intermediate values).

Since

$$A' = A \odot (1 + \delta A),$$

the gradient errors in this case are:

$$\begin{aligned} \|\Delta \frac{\partial A}{\partial V}\|_2 &\leq \frac{\|A\|_2}{\|V\|_2} \left(\frac{1}{\sqrt{d}} \|Q \odot \delta Q K^\top + Q(K \odot \delta K)^\top\|_2 + \|A \odot \delta A\|_2 \right) \\ &= \mathcal{O} \left(\frac{\|Q\|_2 \|K\|_2}{\sqrt{d}} (\|\delta Q\|_\infty + \|\delta K\|_\infty) \right). \end{aligned}$$

$$\begin{aligned} \|\Delta \frac{\partial A}{\partial K}\|_2 &\leq \frac{2}{\sqrt{d}} \|V\|_2 \|Q \odot \delta Q K^\top + Q(K \odot \delta K)^\top\|_2 \\ &\quad + 2\|V\|_2 \|\delta V\|_\infty + \|A \odot \delta A\|_2 \\ &= \mathcal{O} \left(\frac{\|Q\|_2 \|K\|_2^2 \|V\|_2}{d} (\|\delta Q\|_\infty + \|\delta K\|_\infty) \right). \end{aligned}$$

$$\begin{aligned} \|\Delta \frac{\partial A}{\partial Q}\|_2 &\leq \frac{1}{\sqrt{d}} \left[\frac{2}{\sqrt{d}} \|V\|_2 \|K\|_2 \|Q \odot \delta Q K^\top + Q(K \odot \delta K)^\top\|_2 \right. \\ &\quad + 2\|V\|_2 \|K\|_2 \|\delta V\|_\infty \\ &\quad \left. + \|K\|_2 \|A \odot \delta A\|_2 + \|V\|_2 \|\delta K\|_\infty \right] \\ &= \mathcal{O} \left(\frac{\|V\|_2 \|Q\|_2^2 \|K\|_2}{d} (\|\delta Q\|_\infty + \|\delta K\|_\infty) \right). \end{aligned}$$

Based on the derived error bounds under the ‘‘recompute intermediate values’’ case (Case 1), we compare the gradient perturbations of RMSNorm and the attention operation. According to Eq. 15 and the subsequent analysis, the RMSNorm gradient error satisfies $\|\Delta J\|_2 = \mathcal{O}(\eta)$ under standard Transformer assumptions ($\|X\|_2^2 = \Theta(d)$, $r = \Theta(1)$, and $\|\gamma\|_\infty = \Theta(1)$), where $\eta = \|\delta X\|_\infty$ is the normalized input perturbation level. In contrast, the attention gradient errors scale much more severely with the sequence length L and the per-head dimension d_k . For $\partial A / \partial V$ we obtain $\|\Delta(\partial A / \partial V)\|_2 = \mathcal{O}(\eta L \sqrt{d_k})$, while for $\partial A / \partial K$

Table 8. GPU and NPU Memory Usage (GB) Comparison

GPU/NPU	AGoQ	O+G	O	Megatron-LM
GPU	22.3	35.3	37.7	46.1
NPU	29.7	40.5	45.3	55.2

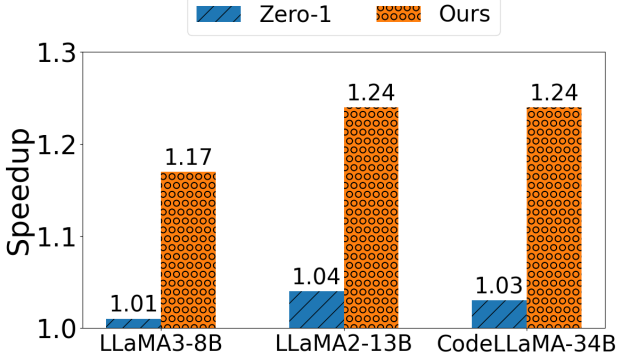


Figure 11. Speedups of our AGoQ and ZeRO-1 over Megatron-LM on varied models.

and $\partial A/\partial Q$ the bounds grow to $\mathcal{O}(\eta L^{3/2} \sqrt{d_k})$. These quantities are larger than the RMSNorm error by factors of $\Theta(L\sqrt{d_k})$ and $\Theta(L^{3/2}\sqrt{d_k})$, respectively. Such a multiplicative gap explains why activation quantization applied to the Q, K, V projections leads to severely amplified gradient errors and training instability, whereas RMSNorm can be safely quantized. **Therefore, for the attention operation, we choose not to apply activation quantization.**

Table 9. Experimental configurations.

# of GPUs	Model	Seq. Len.	TP	PP
8	LLaMA3-8B	16K–32K	4	1
8	LLaMA3-8B	32K–80K	8	1
16	LLaMA2-13B	32K–80K	8	[1,2]
32	LLaMA2-13B	32K–80K	8	[1,2,4]
32	CodeLLaMA-34B	32K–80K	8	[1,2,4]
64	LLaMA2-13B	32K–80K	8	[1,2,4,8]
64	CodeLLaMA-34B	32K–80K	8	[1,2,4,8]

8.2. Ablation Studies

Performance on Varied Models. To evaluate improvements in our methods across different models, we organize the experiments according to the setup in Table 9. Specifically, we compare the speedup of Zero-1 and AGoQ against Megatron-LM on LLaMA3-8B, LLaMA2-13B, and CodeLLaMA-34B, as shown in Fig. 11. The results demonstrate that our method consistently delivers substantial gains across all models. Detail configurations are shown in Table 9.

Zero-shot Accuracy. We further assess zero-shot accuracy of LLaMA2-7B trained on 2B tokens and LLaMA3.2-1B

 Table 10. Accuracy changes (Δ Acc) of AGoQ compared to the Megatron-LM FP16 baseline across datasets. Results are shown for LLaMA2-7B (2B tokens) and LLaMA3.2-1B (10B tokens).

Dataset	LLaMA2-7B (2B tokens)			LLaMA3.2-1B (10B tokens)		
	Baseline	AGoQ	Δ	Baseline	AGoQ	Δ
arc.c	0.1988	0.1834	-0.0154	0.1877	0.2099	+0.0222
arc.e	0.4179	0.4158	-0.0021	0.4571	0.4714	+0.0143
hellas.	0.2886	0.2897	+0.0011	0.3276	0.3298	+0.0022
piqa	0.5990	0.6039	+0.0049	0.6219	0.6284	+0.0065
sciq	0.7260	0.7280	+0.0020	0.7180	0.7100	-0.0080
winog.	0.4830	0.5036	+0.0206	0.5193	0.5185	-0.0008

Table 11. Speedups of kernel fusion of GEMM and quantization/dequantization over sequential operations.

m	k	n	Speedup
16,384	14,336	4,096	1.08x
16,384	4,096	14,336	1.03x
16,384	4,096	12,288	1.04x
16,384	4,096	4,096	1.11x

tained on 10B tokens using ARC-Challenge (Clark et al., 2018), ARC-Easy (Clark et al., 2018), HellaSwag (Clark et al., 2018), PIQA (Bisk et al., 2020), SciQ (Welbl et al., 2017), and Winogrande (Sakaguchi et al., 2021). Table 10 shows that the mean accuracy across all six datasets remains high with no degradation.

Memory Footprint Reduction. We also compare the effects of activation quantization, gradient quantization, and optimizer quantization on memory reduction in Table 8 with TP=8, PP=1, sequence length of 12K on LLaMA2-13B with GPUs and NPUs. As can be seen from the table, each quantization module contributes significantly to reducing memory usage. Specifically, only applying 8-bit optimizer quantization (“O”) (Dettmers et al., 2022), it saves around 8G memory size on GPU and 10G on NPU, while our activation and gradient quantization further save 13G and 2.4G memory on GPU while 10.8G and 4.8G on NPU, respectively. Notably, our AGoQ (“A+O+G”) achieves a 53% reduction of the peak memory footprint over Megatron-LM on GPU while 46% on NPU.

Improvement of kernel fusion. We test the speedup achieved by kernel fusion (§4.3) of GEMM computation with quantization or dequantization operations. As shown in Table 11, we demonstrate several major matrix shapes for GEMM from LLaMA3-8B as examples, where m, k, n represent the dimensions of GEMM $C = A \times B$, where $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$. The results demonstrate that our kernel fusion approach achieves an average speedup of $1.07\times$ over the sequential version of GEMM and quantization/dequantization, which results in an end-to-end speedup of $1.05\times$ on LLaMA3-8B with PP=1, TP=8 and sequence length of 16K.

Ablation for DBC (Dynamic Block Compression). We conducted an ablation study comparing training with and without DBC. Table 12 shows that DBC consistently improves accuracy on most tasks. Notably, DBC configured under low pipeline parallelism (PP) can be applied to high PP without increasing peak memory, highlighting its flexibility.

Table 12. Ablation study of DBC: accuracy (%) with and without DBC on six datasets.

	arc_c	arc_e	hellas.	piqa	sciq	winog.
w/o DBC	17.66	41.88	28.53	60.12	71.20	50.83
w/ DBC	18.34	41.58	28.97	60.39	72.80	50.36

Ablation for Layer-Aware Bit Width. We compared our layer-aware mixed-precision quantization against a naïve uniform 4-bit quantization applied to all activations. The uniform 4-bit baseline failed to converge, underscoring the importance of adaptive precision.

Actual Per-Layer Gradient Error. We measured the gradient error of input activations for each layer (mean absolute error and normalized L2 distance) introduced by AGoQ compared to full-precision gradients, sampled from a model trained after 10B tokens. For GEMM, we additionally compare the gradient error of the weight. Table 13 reports the results. The normalized L2 errors for Attention (0.14–0.15) are the largest among all layers, which aligns with our theoretical analysis and explains why we exclude Attention from quantization. For other layers, the minimum normalized error is 0.003 (LayerNorm) and the maximum is 0.051 (SiLU), both of which have negligible impact on convergence.

Table 13. Per-layer gradient error (mean absolute error and normalized L2 distance) introduced by AGoQ.

Layer	MAE	Normalized L2
LayerNorm	2.8×10^{-10}	0.003
GEMM (Weight)	5.3×10^{-7}	0.026
SiLU	2.6×10^{-9}	0.051
Attention Q	2.5×10^{-9}	0.14
Attention K	1.7×10^{-9}	0.15
Attention V	2.0×10^{-9}	0.059

Memory Reduction at 32k/64k Sequence Lengths. We conducted memory usage experiments on Llama3-8B with extended sequence lengths under memory constraints. For 32k sequence length, we configured 36 layers; for 64k, we used 16 layers due to memory limitations. As shown in Table 14, AGoQ achieves substantial memory savings, reducing footprint by up to 66% at 32k and 59% at 64k, demonstrating its effectiveness in memory-constrained long-sequence scenarios.

Gradient Norm Monitoring. We conducted an extended

Table 14. Memory consumption (MB) for Llama3-8B at 32k and 64k sequence lengths.

Sequence Length	Baseline (MB)	AGoQ (MB)
32k (36 layers)	48606	16594
64k (16 layers)	46681	19267

training run on Llama3-8B with a per-iteration token budget of 32k, spanning 100,000 iterations. Table 15 reports average gradient norms at 10k-iteration intervals. AGoQ maintains gradient norms closely aligned with the baseline throughout training, with stable convergence.

Table 15. Gradient norm comparison every 10k iterations over 100k iterations.

Iteration Range	AGoQ	Baseline
0–10k	7.28	7.10
10k–20k	4.33	4.38
20k–30k	3.79	3.77
30k–40k	3.64	3.36
40k–50k	3.38	3.20
50k–60k	3.32	3.10
60k–70k	3.17	3.15
70k–80k	3.32	3.29
80k–90k	3.20	3.37
90k–100k	3.08	3.21